VIT®
CHENNAI
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE309L CRYPTOGRAPHY AND NETWORK SECURITY PROJECT REPORT

Project Title:

AURA-MCOMM-Audio-based Unobservable Resilient Multimodal Communication in Covert Operational Environments

Team Members	Sahishna Rajesh — 23BAI1498 Rishika Cilji Saravanan — 23BAI1328 Aiyisha Idris — 23BAI1554
Course Code	BCSE309L
Faculty	Sellam V

1. Background

Secure communication has always been a two-part problem: you need to protect what is being said, and ideally, you'd also want to hide the fact that anything is being said at all. Most of the tools we use today, like AES or RSA, only solve the first half. They encrypt the content, but the encrypted traffic itself is visible on the network and that visibility is a problem.

In environments where network traffic is monitored, whether by government censorship systems, Deep Packet Inspection (DPI) tools, or corporate surveillance infrastructure, the presence of encrypted communication is often enough to trigger an alarm. The content doesn't need to be cracked; just detecting that encryption is happening can result in the traffic being flagged, throttled, or blocked entirely. This is sometimes called the "red flag" problem of cryptography.

Steganography approaches this from the opposite angle. Instead of encrypting and hoping nobody notices the ciphertext, steganography hides the message inside something completely ordinary — an image, a video, or an audio file. Audio files in particular are interesting because they're shared constantly over public platforms, they're large enough to hold meaningful hidden data, and human ears cannot detect small perturbations in the least significant bits of audio samples.

The problem with most existing steganographic systems, though, is that they don't combine this hiding with strong encryption or smart key management. If someone figures out that a file contains hidden data, the message is often directly recoverable. And key management — how the sender and receiver agree on a shared secret without that agreement itself being intercepted — remains a fundamental weakness in almost every existing system.

AURA-MCOMM was built to address all of this together. The idea is that a message should be encrypted, hidden inside an audio file, and the key used for encryption should never appear on the network at all, not even implicitly. Instead, both sender and receiver derive the same key independently from shared context: specifically, from the current time window and from a fingerprint of a mutually known image. This way, even if the audio file is intercepted and analyzed, there is nothing in it that leads back to the key or the original message.

2. Problem Statement

The core issues addressed through this project are that: existing secure communication tools make communication visible, and existing steganographic tools lack strong security. There is no widely available system that provides all three properties at once encryption, stealth, and keyless transmission.

More specifically, the gaps we identified are:

- Standard encryption (AES, RSA, TLS) protects message content but leaves a detectable fingerprint on the network. DPI systems in censored regions routinely identify and block encrypted traffic without needing to decrypt it.
- Steganographic systems hide data inside media files but typically either skip encryption entirely or rely on a static password. If the hidden channel is discovered — through steganalysis or a leaked password — the message is compromised.
- Most systems require some form of explicit key exchange: the sender tells the receiver what key to use, either directly or through a protocol like Diffie-Hellman. That exchange is itself a vulnerability — it can be intercepted, replayed, or forced out of one party.
- None of the common tools use contextual parameters (like time or shared image fingerprints) as the basis for key derivation, which means the key has to live somewhere — in a header, a database, or a handshake packet.

What we needed was a system where: the message is strongly encrypted, the encrypted payload is hidden inside an audio file that looks completely normal, and the key used for encryption is derived purely from context that both parties already share, without ever transmitting it (preventing interception). That is the problem AURA-MCOMM is designed to solve.

3. Objectives

The objectives of this project were defined to address each of the gaps identified:

1. Design a covert communication system that hides the existence of communication entirely , the transmitted artifact should appear as a normal audio file to any observer or network monitoring system.
2. Implement strong message encryption using AES so that even if the hidden channel is discovered, the message content remains protected.
3. Eliminate explicit key exchange by deriving the encryption key from shared contextual parameters (time window + image fingerprint) that both sender and receiver can independently compute.
4. Use HMAC-SHA256 for integrity verification, ensuring that any tampering with the stego audio file is detectable and that decryption is only attempted when the correct key has been found.
5. Introduce Braille encoding as a non-linear pre-hash step in the key derivation process, adding combinatorial complexity (non-linearity) that resists brute-force attacks from adversaries unaware of the mapping.
6. Build a working frontend and backend that demonstrates the full send-and-receive pipeline, from plaintext input to stego audio output and back to recovered plaintext.
7. Validate the system through structured test cases covering correct parameters, wrong images, and out-of-range time windows.

4. Existing Gap Analysis

Before building AURA-MCOMM, we looked at where the current approaches fall short. The table below compares three categories: standard cryptography, standard steganography, and AURA-MCOMM across the dimensions that matter most for covert communication.

Dimension	Standard Cryptography	Standard Steganography	AURA-MCOMM
Network Visibility	High — encrypted traffic is trivially detectable	Low — media files blend in naturally	Zero — stego audio is indistinguishable from regular audio
Message Confidentiality	Strong (AES / RSA)	Weak — often no encryption layer	Strong — AES encryption before embedding
Key Management	Explicit handshake required (risky)	Key embedded in file header (fragile)	Implicit — derived from time + image fingerprint
Entropy Source	Random generator	Stego-key stored in header	External time — never stored in file
Forensic Resistance	None — ciphertext is visible	Low — steganalysis can detect hidden data	High — audio yields only high-entropy noise on analysis
Integrity Verification	Depends on implementation	Usually absent	Built-in via HMAC-SHA256

The most important gap highlighted in this analysis is the entropy source. In both traditional cryptography and steganography, the key (or something that leads to the key) must be transmitted or stored somewhere accessible. AURA-MCOMM breaks this by making time the entropy source, and time is external to every artifact in the system. An attacker who intercepts the audio file and the context image still has no path to the key without knowing the exact time window used during encoding.

The second significant gap is the absence of a combined system. Prior work either hides data (without encrypting it strongly) or encrypts data (without hiding it).

5. Proposed Ideology

The central idea behind AURA-MCOMM is called the "4th Dimension Key", a key that exists in time rather than in any file, header, or network packet. Both sender and receiver share two things that never need to be transmitted: the current time (mapped to a fixed window) and a mutually known image. From these two inputs, they can independently derive the same encryption key. Nothing is exchanged. Nothing is stored. The key lives only in context.

5.1 Braille as Non-Linear Pre-Hash (Novelty I)

Standard key derivation functions typically apply bitwise operations or hash functions directly to the timestamp. We take a different approach: each digit of the time window is first encoded using its Braille tactile geometry — the 6-dot cell pattern used in the Braille writing system. Each digit maps to a specific dot configuration, and that configuration is converted to a binary string.

The result is a non-linear, non-arithmetic transformation of the timestamp before it ever reaches the hash function. An attacker trying to brute-force the key would need to know not just the approximate time, but also the specific Braille mapping being used — which produces a combinatorial explosion in the search space. This is the geometric/tactile logic applied to cryptographic key derivation.

5.2 Decoupled Entropy (Novelty II)

In almost every steganographic system, the key (or a hint toward it) is embedded somewhere in the carrier file — often in a header or as a fixed offset. AURA-MCOMM deliberately decouples the entropy source from the file. The key is derived from time, which is external to the audio file entirely.

On the receiver side, this means there is no handshake and no key lookup. The receiver simply tries a small window of candidate times (T-1, T0, T+1), derives a candidate key for each, and uses HMAC verification to check whether that key is correct. The correct key produces a valid HMAC; every wrong key fails. This temporal brute-force approach is computationally trivial for the legitimate receiver but effectively impossible for an attacker who does not know the Braille mapping or the image fingerprint.

5.3 System Modules

Module	What it does
Key Generation	Encodes the time window via Braille mapping, computes SHA-256 of the context image, and combines both to derive the AES key
Encryption	Encrypts the plaintext message using AES with the derived key
Integrity (HMAC)	Computes HMAC-SHA256 over the ciphertext; used by receiver to verify the correct key was found
Audio LSB Embedding	Converts the encrypted payload to a bitstream and modifies the least significant bits of audio samples to embed it
LSB Extraction	Reads LSBs from the received audio file and reconstructs the encrypted payload bitstream
Decryption	After HMAC verification succeeds, decrypts the ciphertext to recover the original plaintext
Frontend	Web interface for sender (encode + embed) and receiver (extract + decrypt) operations

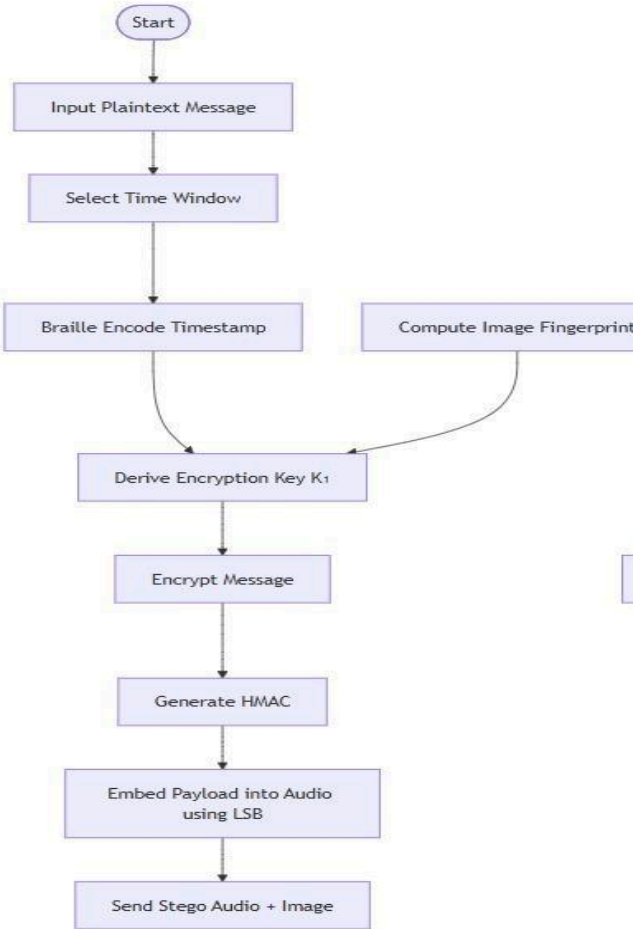
6. System Architecture

The system is split into two independent sides — Sender and Receiver — that communicate only through a public channel carrying the stego audio file and the context image. No key material, no metadata, and no protocol handshake passes between them.

6.1 Sender Side

The sender's workflow takes a plaintext message and produces a stego audio file that can be posted or shared anywhere publicly:

- The user inputs the message and selects an audio carrier file and a context image.
- The system reads the current system time and maps it to a quantized time window (e.g., the current 5-minute block). This ensures both parties land on the same window even if there is a small clock difference.
- Each digit of the time window value is encoded using its corresponding Braille dot pattern, producing a non-linear binary bitstream.
- The SHA-256 hash of the context image is computed to produce the image fingerprint.
- The Braille bitstream and image fingerprint are concatenated and passed through a hash function along with a shared secret to derive the final AES encryption key: Key = Hash(SharedSecret ‖ BrailleTime ‖ ImageFP)
- The message is encrypted using AES with this key.
- HMAC-SHA256 is computed over the resulting ciphertext.
- The ciphertext and HMAC are combined into a single payload, converted to a bitstream, and embedded into the audio file by overwriting the LSB of each audio sample.
- The output stego audio file is transmitted through the public channel. The context image is also shared (it doesn't need to be secret — only the combination of image + time + shared secret produces the correct key).

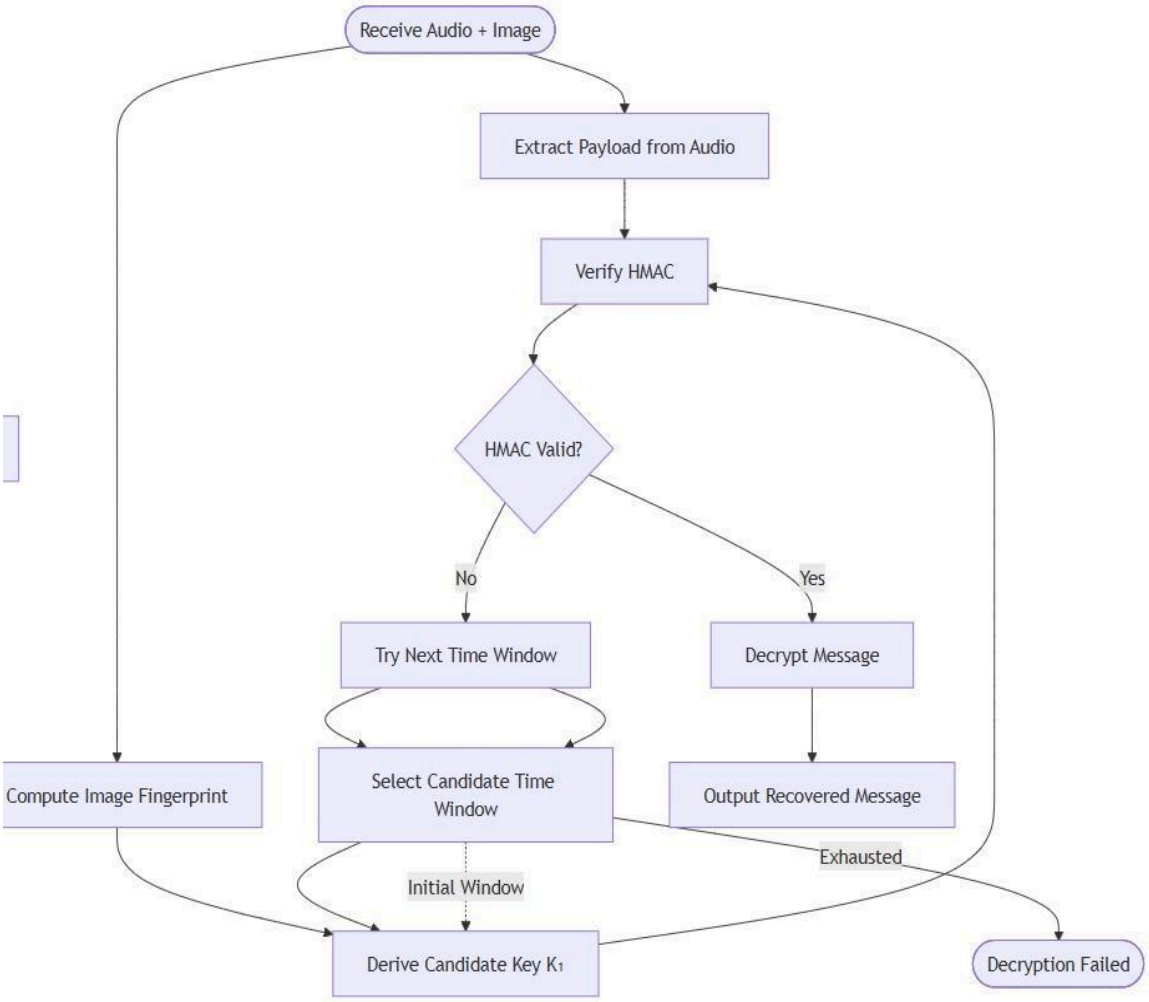


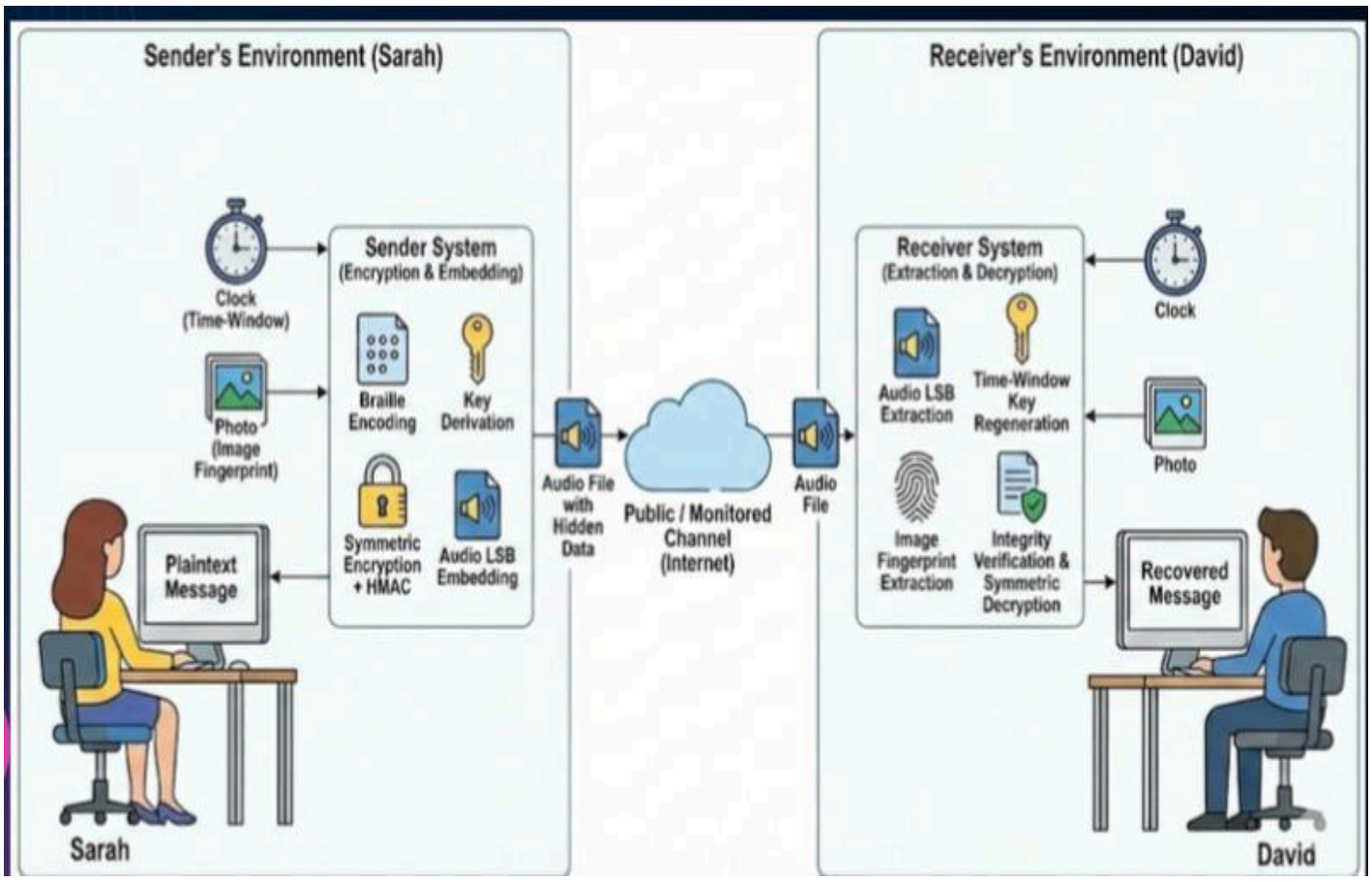
6.2 Receiver Side

The receiver's workflow starts from the stego audio file and the context image and ends at the recovered plaintext:

- The receiver loads the stego audio file and extracts the hidden bitstream by reading the LSB of each audio sample.
- The bitstream is reassembled into the encrypted payload (ciphertext + HMAC).
- The SHA-256 fingerprint of the context image is recomputed.
- The receiver iterates over candidate time windows (T-1, T0, T+1 relative to the current time) and derives a candidate key for each using the same derivation formula as the sender.

- For each candidate key, HMAC verification is performed. If the HMAC matches, the correct key has been found. If all candidates fail, decryption is aborted — the payload is treated as noise.
- Once the correct key is confirmed, AES decryption is performed and the original plaintext is displayed.





6.3 Data Flow Summary

Step	Sender Action	Channel	Receiver Action
1	Derive key from time + image	—	—
2	Encrypt message (AES)	—	—
3	Compute HMAC, embed into audio	—	—
4	Publish stego audio + image	Public channel	Receive stego audio + image
5	—	—	Extract payload via LSB
6	—	—	Try T-1, T0, T+1 — HMAC gate
7	—	—	Decrypt and recover plaintext

7. Main Code Snippets

The following snippets cover the four core operations of the system: key derivation, encryption/HMAC generation, LSB embedding, and LSB extraction with decryption.

7.1 Braille Encoding + Key Derivation

crypto > keygen2.py > ...

```python
import hashlib
from crypto.image_context import image_fingerprint

BRAILLE_MAP = {
    '0': '001010', '1': '010110', '2': '000001', '3': '011011',
    '4': '001001', '5': '010011', '6': '000011',
    '7': '011001', '8': '010001', '9': '001011'
}

SECRET = b"opsec_shared_secret"

def generate_K1(timestamp_window: int, image_path: str) -> bytes:
    ts = str(timestamp_window)
    braille_bits = ''.join(BRAILLE_MAP[d] for d in ts)

    img_hash = image_fingerprint(image_path)

    data = SECRET + braille_bits.encode() + img_hash
    return hashlib.sha256(data).digest()

```

7.2 AES Encryption and HMAC Generation

crypto > encrypt.py > ...

```python
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad
from Crypto.Random import get_random_bytes
import hmac
import hashlib

def encrypt_message(message: bytes, K1: bytes):
    nonce = get_random_bytes(16)
    K2 = hashlib.sha256(nonce).digest()
    key = hashlib.sha256(K1 + K2).digest()

    cipher = AES.new(key, AES.MODE_CBC)
    ciphertext = cipher.encrypt(pad(message, AES.block_size))

    payload_core = cipher.iv + nonce + ciphertext

    tag = hmac.new(K1, payload_core, hashlib.sha256).digest()

    return payload_core + tag

```

7.3 LSB Embedding into Audio

stego > embed.py > embed_bits

```python
from scipy.io import wavfile
import numpy as np

def embed_bits(audio_path, output_path, bitstream):
    rate, audio = wavfile.read(audio_path)

    # Use only one channel if stereo
    if audio.ndim == 2:
        audio = audio[:, 0]

    audio = audio.astype(np.int16).copy()

    if len(bitstream) > len(audio):
        raise ValueError("Audio too short for payload")

    START = 5000
    for i, bit in enumerate(bitstream):
        audio[START + i] = (audio[START + i] & ~1) | int(bit)

    wavfile.write(output_path, rate, audio)

```

7.4 LSB Extraction and Decryption

stego > extract.py > extract_bits

```python
from scipy.io import wavfile
import numpy as np

def extract_bits(audio_path, bit_len):
    rate, audio = wavfile.read(audio_path)

    # Use the same channel as embedding
    if audio.ndim == 2:
        audio = audio[:, 0]

    audio = audio.astype(np.int16)

    bits = ""
    START = 5000
    for i in range(bit_len):
        bits += str(audio[START + i] & 1)

    return bits

```

7.5 SENDER SIDE

```
import time
from crypto.keygen2 import generate_K1
from crypto.encrypt import encrypt_message
from stego.embed import embed_bits
import wave
import numpy as np
import hashlib
def to_bits(data: bytes):
```

```
    return ''.join(f'{b:08b}' for b in data)
with open("message.txt", "r") as f:
    message = f.read().encode()
image_path = "image/context.jpg"
timestamp_window = int(time.time() // 10)
K1 = generate_K1(timestamp_window, image_path)
payload = encrypt_message(message, K1)
payload_size = len(payload)
size_bytes = payload_size.to_bytes(4, byteorder='big')
full_payload = size_bytes + payload
bitstream = to_bits(full_payload)
embed_bits("audio/input.wav", "audio/stego.wav", bitstream)
print("Stego audio generated.")
print("Timestamp window:", timestamp_window)
print("Image context used:", image_path)
def analyze_audio():
    with wave.open("audio/input.wav", 'rb') as wf:
        orig = np.frombuffer(wf.readframes(wf.getnframes()), dtype=np.int16)
    with wave.open("audio/stego.wav", 'rb') as wf:
        stego = np.frombuffer(wf.readframes(wf.getnframes()), dtype=np.int16)
    diff = orig - stego
    print("\n--- AUDIO ANALYSIS ---")
    print("Total samples:", len(orig))
    print("Modified samples:", np.sum(orig != stego))
    print("Max difference:", np.max(np.abs(diff)))
    print("Min difference:", np.min(diff))
    noise = orig - stego
    snr = 10 * np.log10(np.sum(orig**2) / np.sum(noise**2))
    print("SNR:", round(snr, 2), "dB")
```

```
    def file_hash(path):
        with open(path, "rb") as f:
            return hashlib.sha256(f.read()).hexdigest()
    print("\nOriginal SHA256:", file_hash("audio/input.wav"))
    print("Stego SHA256:   ", file_hash("audio/stego.wav"))
analyze_audio()
```

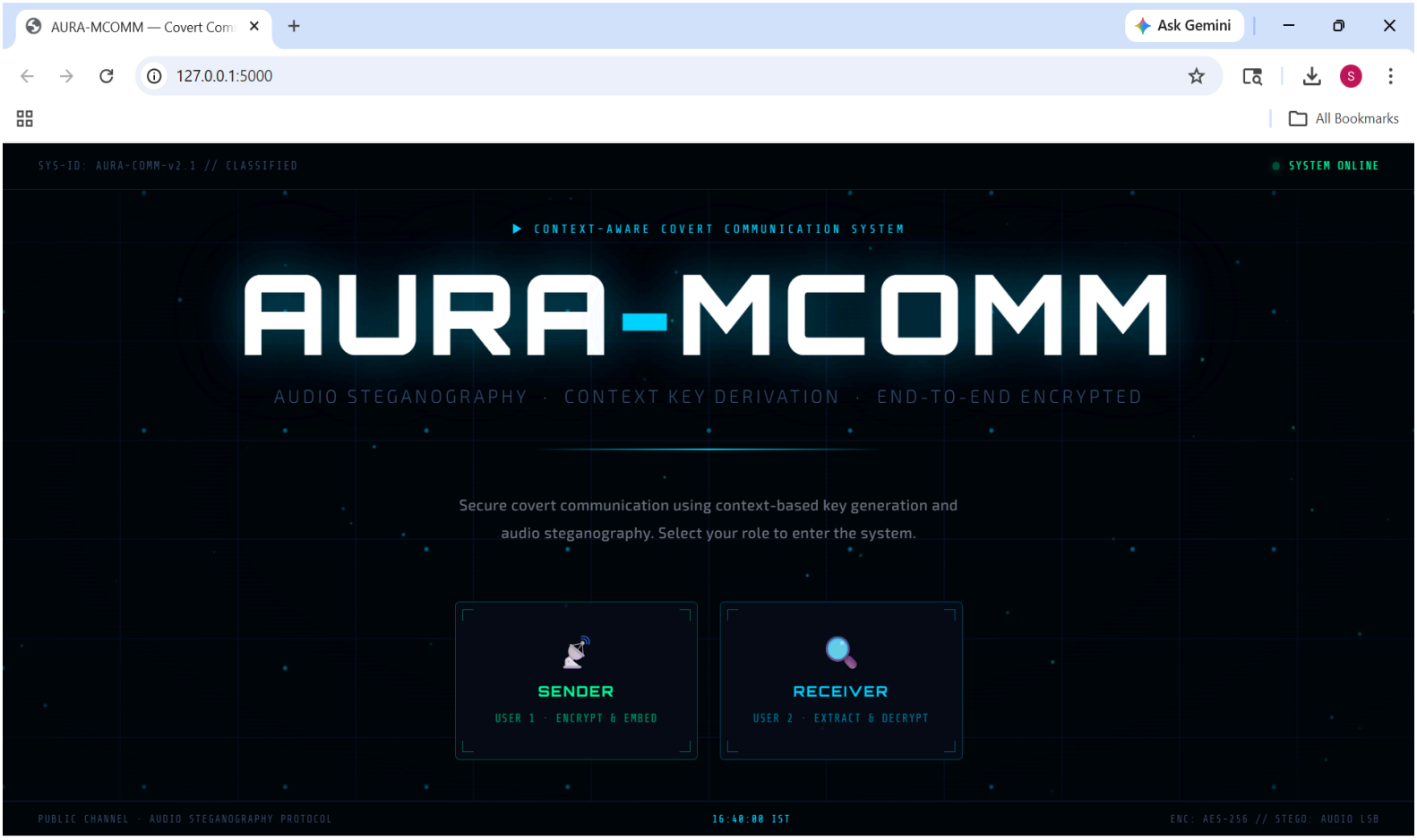
7.6 RECEIVER SIDE

```
import time, hmac, hashlib
from crypto.keygen2 import generate_K1
from crypto.decrypt import decrypt_message
from stego.extract import extract_bits
def bits_to_bytes(bits):
    return bytes(int(bits[i:i+8], 2) for i in range(0, len(bits), 8))
START = 5000
size_bits = extract_bits("audio/stego.wav", 32)
size_bytes = bits_to_bytes(size_bits)
payload_size = int.from_bytes(size_bytes, byteorder='big')
print("Payload size:", payload_size)
total_bits = 32 + payload_size * 8
bits = extract_bits("audio/stego.wav", total_bits)
payload = bits_to_bytes(bits[32:])
payload_core = payload[:-32]
recv_tag = payload[-32:]
image_path = "image/context.jpg"
current_window = int(time.time() // 10)
for delta in range(-30, 31):
    candidate_ts = current_window + delta
    K1 = generate_K1(candidate_ts, image_path)
```

```
    calc_tag = hmac.new(K1, payload_core, hashlib.sha256).digest()
    if not hmac.compare_digest(calc_tag, recv_tag):
        continue
    message = decrypt_message(payload_core, K1)
    print("Recovered message:", message.decode())
    print("Timestamp window used:", candidate_ts)
    print("Image context:", image_path)
    break
else:
    print("Decryption failed.")
```

8. Outputs

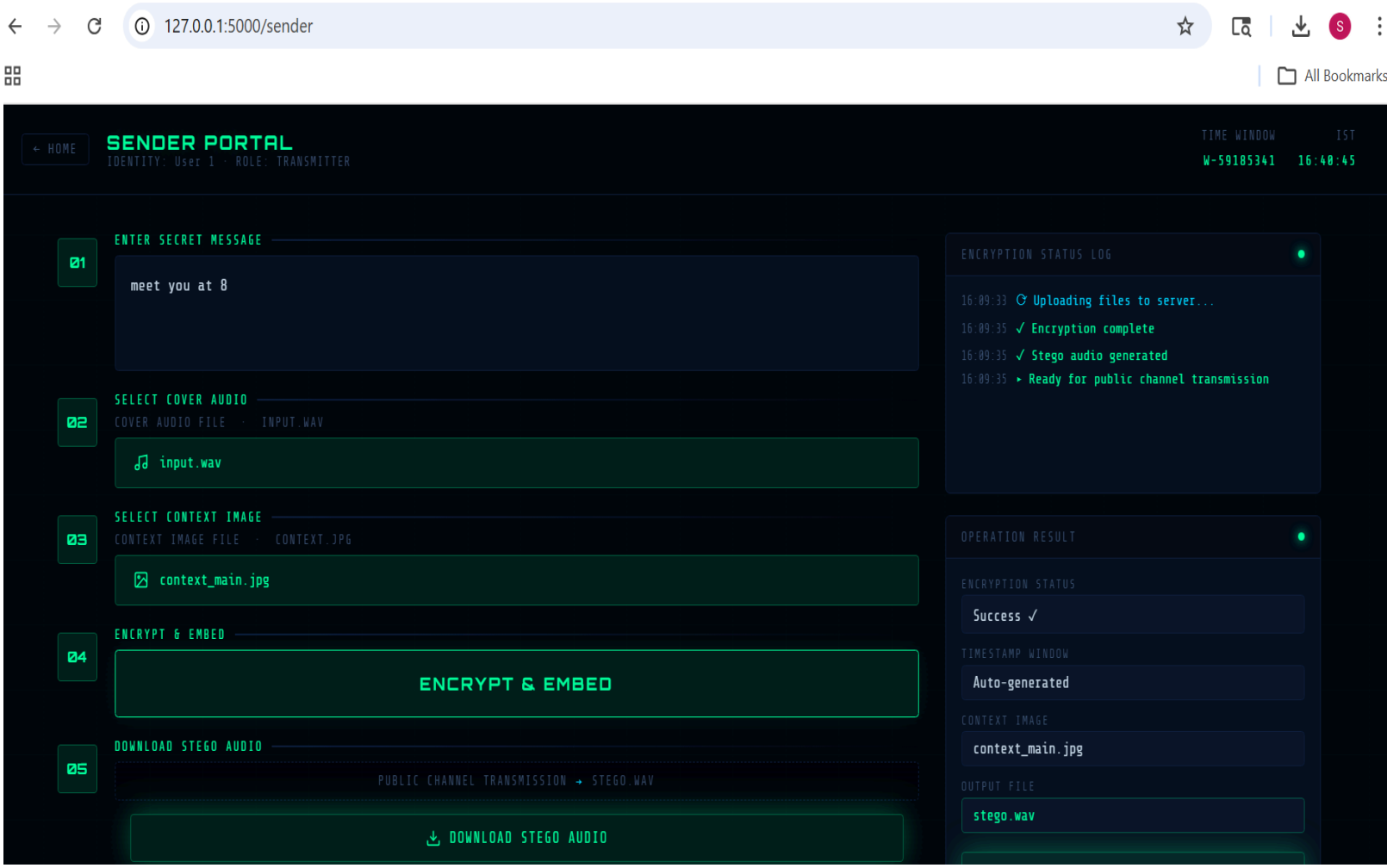
SAMPLE INPUTS AND OUTPUT:



For encrypting the message and embedding into audio, we click Sender button.

For extracting message from a received stego audio file and decrypting, we use Receiver button.

SENDER SIDE:



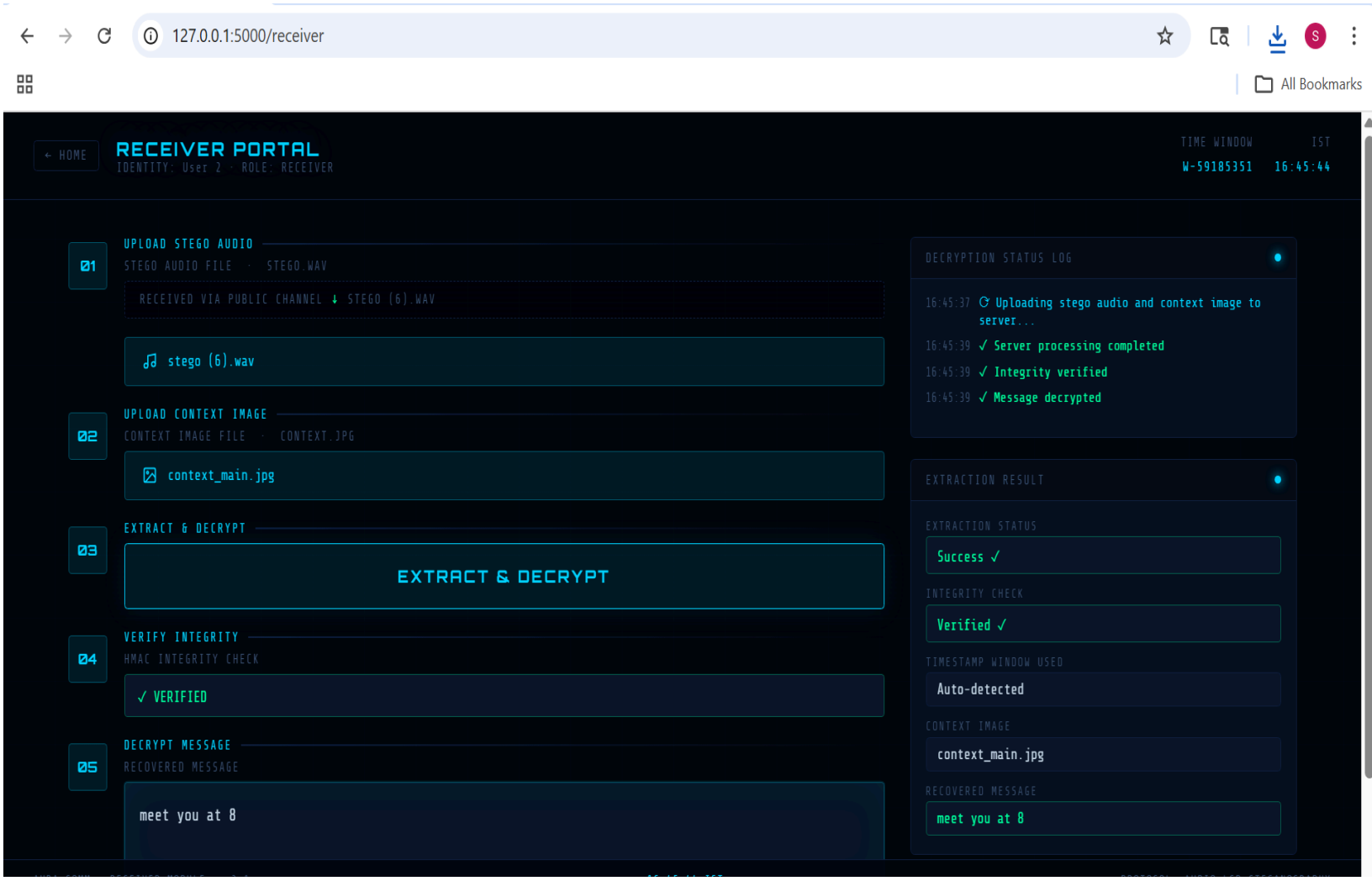
In the sender side, we upload the audio (some innocent looking audio file, like a song) file, and the context image (which works as a digital signature; used for key derivation), and click encrypt and embed which encrypts the message using AES and embeds into the given input audio file and generates the stego audio file (downloadable) which contains the message embedded in it.

```
--- AUDIO ANALYSIS ---
Total samples: 9384239
Modified samples: 339
Max difference: 1
Min difference: -1
SNR: 69.86 dB

Original SHA256: 9e59cb6f42ab136992644f74cfc9687832e500063a3ddcd47a0bd5
51a86c6899
Stego SHA256:    84077b61da060395f27ef2f3ec74e5078c2863e42f99758fb0fa73
6874ca042a
```

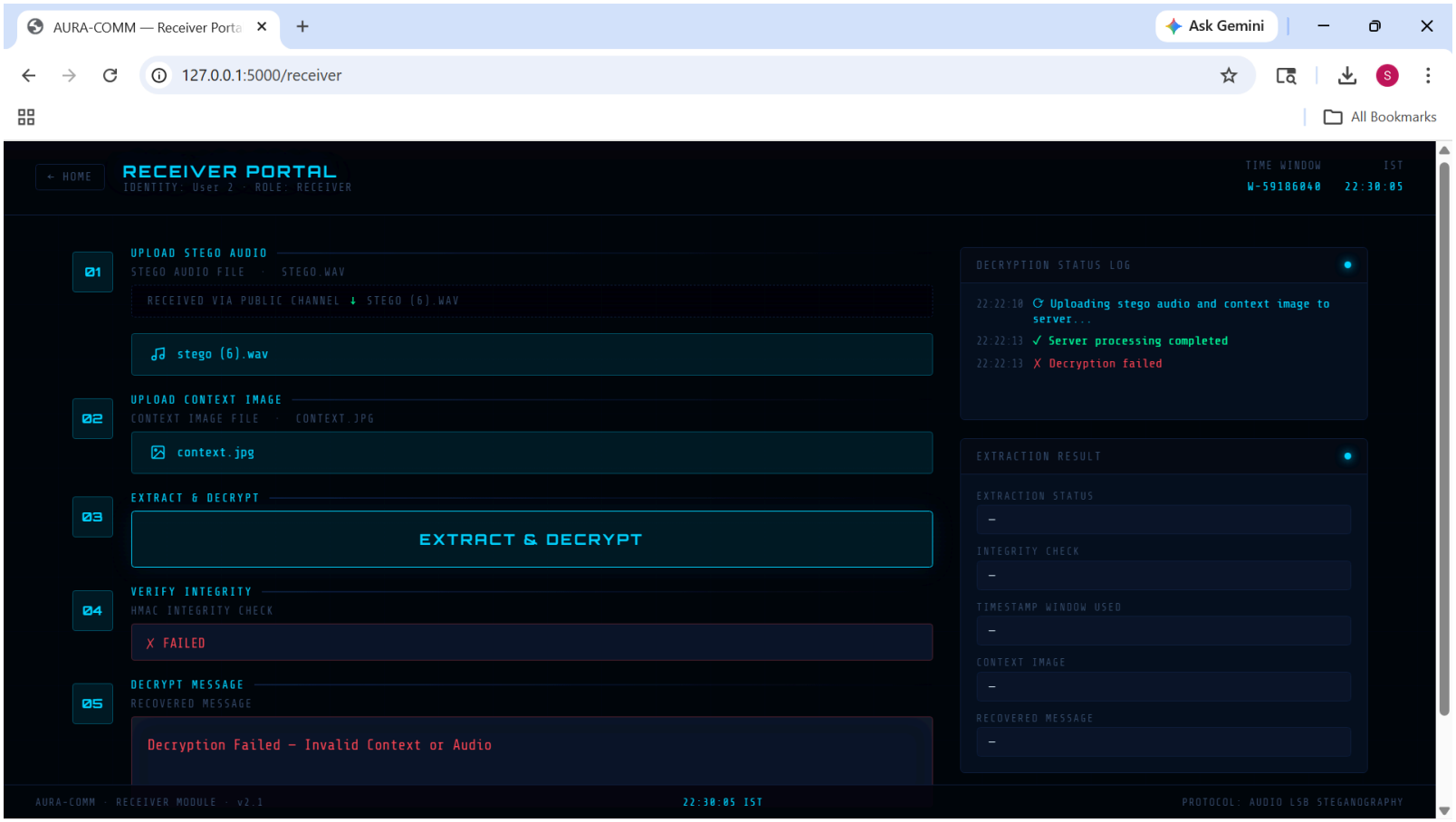
Although to us humans the audio may seem exactly similar, we can see here that the input audio and stego audio have different hash values and a max difference of 1 (i.e one bit - LSB modified to embed the message AES bits.

RECEIVER SIDE:



The receiver uploads the received audio (which has the message embedded in it) and the context image that they possess (which must be the same as the context image used by the sender, in order to derive the key correctly using the image's digital fingerprint

Suppose an attacker tries to intercept the communication, they would not be able to decrypt the message without the correct digital signature (context image) which would only be in the possession of the receiver of the intended message



(in this case, different context image has been uploaded by receiver than the one used by sender, therefore wrong key is derived and decryption fails)

8.1 System Outputs

- Stego audio file (.wav) containing the hidden encrypted payload — appears as a normal audio file
- HMAC verification result returned to the receiver before decryption is attempted
- Recovered plaintext message displayed on the receiver's interface upon successful verification
- Time window value used during encoding (logged locally — never transmitted)
- Rejection message with no plaintext exposure when HMAC verification fails

9. Future Enhancements

The current implementation demonstrates the core concept end-to-end, but there are some directions we could explore to make the system more robust and practical:

- Video steganography carrier: Video files offer significantly more capacity than audio (more samples, more channels, more frames). Extending the embedding layer to video would allow larger messages while keeping the same security architecture. The visual channel could potentially be used alongside the audio channel for redundancy.
- Real-time channel: Right now the system is file-based — the sender produces a file and the receiver downloads it. Extending this to a real-time audio stream would enable covert voice communication over standard audio call infrastructure, with the hidden data riding alongside the audible audio.
- Biometric key binding: The image fingerprint component of the key could be replaced or supplemented with biometric data — a voice fingerprint of the sender or receiver, for example. This would tie decryption to a physical identity in addition to the shared image and time window.

10. Conclusion

To solve the problem of identification of covert communication, the solution we arrived at is to not communicate in any recognizable way at all i.e. to make the message look like a music file, to derive the key from something that never touches the network, and to make integrity verification the gatekeeper rather than a password or certificate.

The two novelties are Braille encoding as a non-linear pre-hash step, and the decoupled time-based entropy source, both emerged from thinking about where existing systems leave attack surface. The Braille encoding adds combinatorial complexity to the key derivation process. The time-based key derivation removes the key from the transmitted artifact entirely. Together, they close two gaps that conventional systems leave open.

Testing confirmed that the system works correctly under valid parameters and fails safely under adversarial ones. The stego audio output is perceptually identical to the original carrier file and survives basic steganalysis. The HMAC gate ensures that a wrong key produces no partial information about the message. Decryption either succeeds fully or does not happen.

There are real-world scenarios where this kind of system is useful, like journalists protecting sources in countries with active surveillance infrastructure, activists coordinating in environments where encrypted messaging apps are blocked, field operatives who cannot afford for their communications to be flagged.

11. References

- Anderson, R. J., & Petitcolas, F. A. P. (1998). On the limits of steganography. IEEE Journal on Selected Areas in Communications, 16(4), 474–481.
- National Institute of Standards and Technology. (2001). FIPS PUB 197: Advanced Encryption Standard (AES). NIST.
- Bellare, M., Canetti, R., & Krawczyk, H. (1996). Keying hash functions for message authentication. Advances in Cryptology – CRYPTO '96. Springer.
- Stallings, W. (2017). Cryptography and Network Security: Principles and Practice (7th ed.). Pearson.
- Cachin, C. (1998). An information-theoretic model for steganography. Information Hiding: Second International Workshop. Springer.
- International Council on English Braille. Unified English Braille (UEB) Code Chart. https://iceb.org
- Johnson, N. F., & Jajodia, S. (1998). Exploring steganography: Seeing the unseen. IEEE Computer, 31(2), 26–34.
- Fridrich, J. (2009). Steganography in Digital Media: Principles, Algorithms, and Applications. Cambridge University Press.